

# Report

## CNN

Convolution is a fundamental operation used in image classification tasks. It extracts local features from an image by applying a **filter (kernel)** that slides across the input. This operation introduces two key properties:

- **Path Independence:**

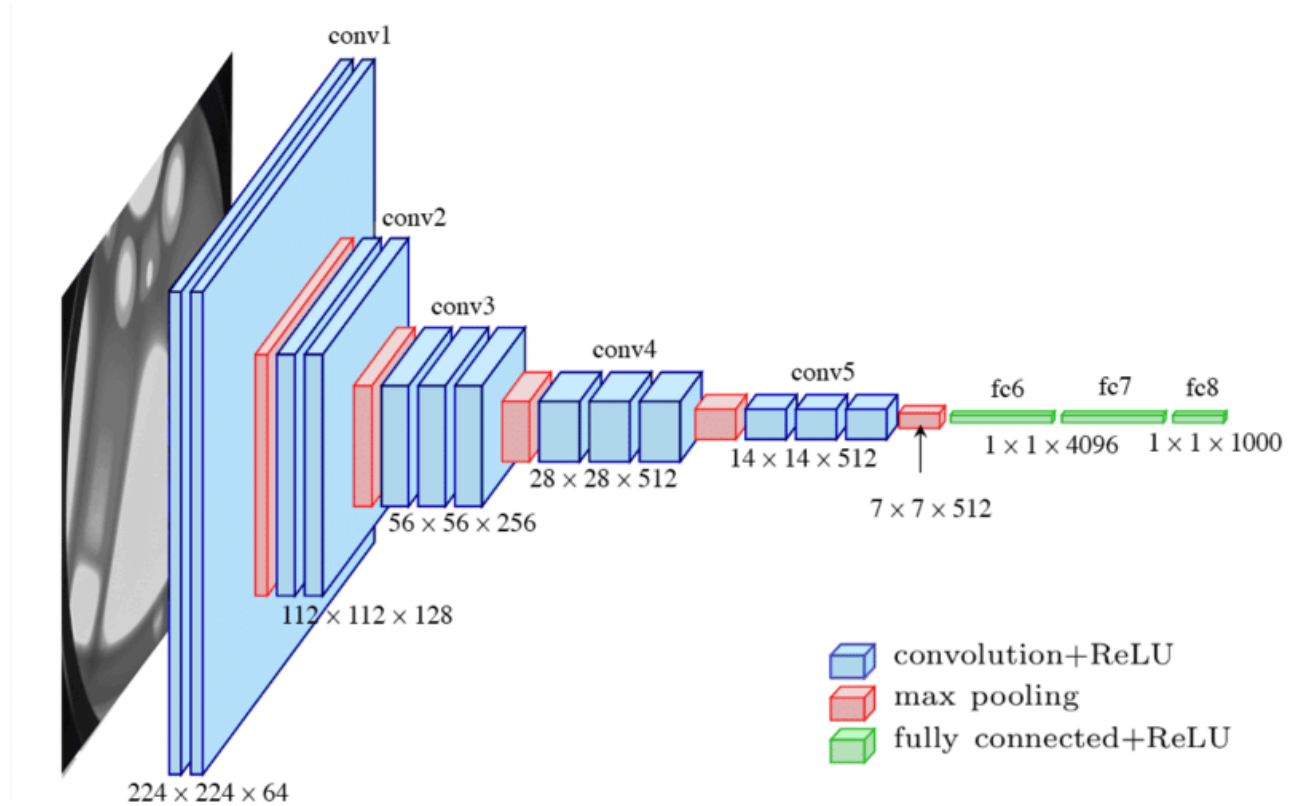
Convolutional layers do not depend on the exact position of features within the image. Instead, they **focus on the presence of patterns** (edges, textures, shapes), making the model **robust to translation and position changes**.

- **Weight Sharing:**

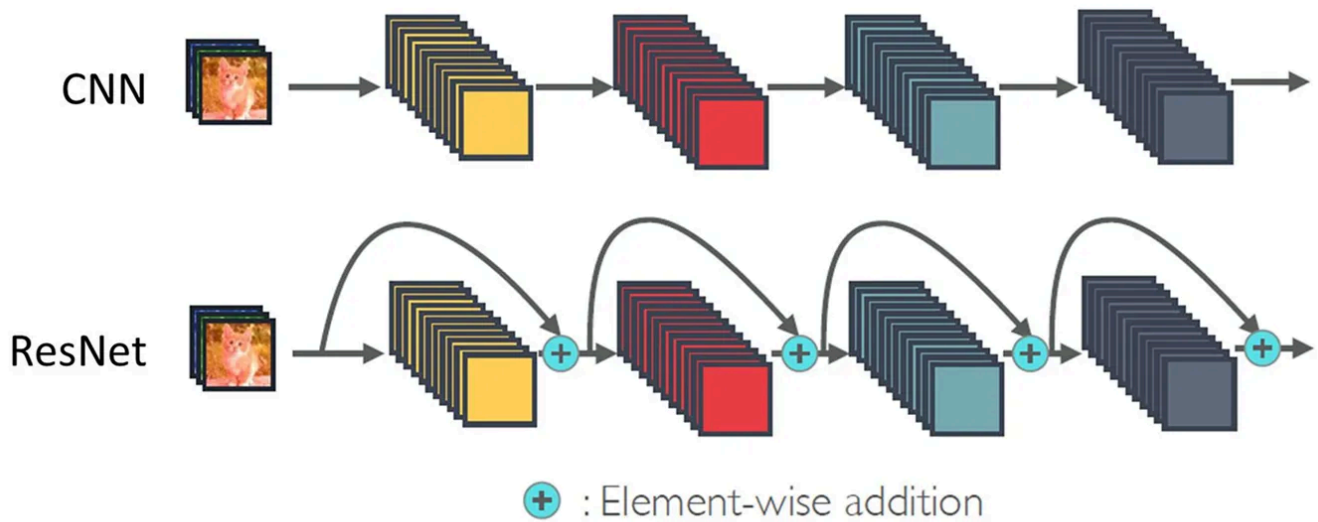
The **same kernel (set of weights)** is applied across all regions of the input image. This drastically reduces the number of parameters compared to fully connected layers, improves generalization, and ensures that the model learns spatially invariant features.

These properties make convolution highly efficient and effective for image classification tasks, as the network learns hierarchical feature representations (from edges in early layers to complex objects in deeper layers).

Some Examples of CNN architectures:



VGG-16 Architecture



ResNet Architecture

# smolCNN

Implementation of CNN in C++

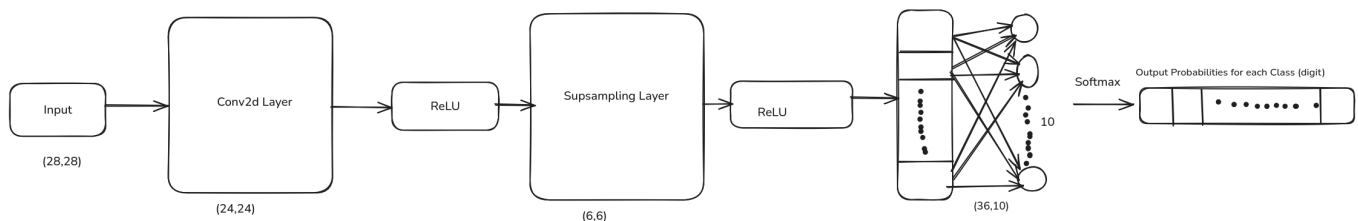
Here I implement a simple version of **LeNet**, one of the first CNN architecture introduced in 1998.

## Why a Simple LeNet Architecture you may ask?

**Simplicity.** Pytorch has way optimized code than whatever I can conjure up in C++, So since it's just for understanding and appreciating the abstractions in Pytorch, this for now is more than plenty.

My architecture is as follows:

- Input
- Conv layer
- ReLU
- Sub-sampling Layer (This was before Max or Mean pooling became a thing)
- ReLU
- MLP
- Softmax



smolCNN architecture

## Architecture - In Depth

A simple CNN with the following layers:

- **Conv2D Layer:**  $1 \rightarrow 6$  channels,  $5 \times 5$  kernel (156 parameters)
- **ReLU Activation**
- **Sub-Sampling Layer** (used as pooling):  $6 \rightarrow 6$  channels,  $4 \times 4$  kernel, stride 4 (486 parameters)
- **ReLU Activation**
- **Flatten Layer**
- **Linear Layer:**  $216 \rightarrow 10$  (2170 parameters)
- **Softmax Activation** (for output probabilities)
- **Total:** ~2812 parameters

## Original PyTorch Implementation

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class smolCNN(nn.Module):
    def __init__(self):
        super(CustomCNN, self).__init__()

        self.conv1 = nn.Conv2d(in_channels=1, out_channels=6, kernel_size=5)
        self.pool = nn.Conv2d(in_channels=6, out_channels=6, kernel_size=4,
stride=4)
        self.fc = nn.Linear(6 * 6 * 6, 10)

    def forward(self, x):
        # Input: (batch_size, 1, 28, 28)
        # Convolution (28x28 -> 24x24)
        x = self.conv1(x) # shape: (batch_size, 6, 24, 24)
        x = F.relu(x)
        # Pooling (custom conv pooling: 24x24 -> 6x6)
        x = self.pool(x) # shape: (batch_size, 6, 6, 6)
        x = F.relu(x)
        # Flatten
        x = x.view(x.size(0), -1) # (batch_size, 6*6*6)
        # Fully connected
        x = self.fc(x) # (batch_size, 10)
        return x
```

### Parameter Count:

- Conv layer:  $6(55+1) = 156$  parameters
- Pooling Layer:  $6(6(4*4+1)) = 486$  parameters
- FC layer:  $216*10 + 10 = 2170$  parameters
- **Total:** ~2812 parameters

---

## PART-1

### Sequential - Implementation

Sequential Implementation of LeNet.

Here there are few important things to implement as seen in pytorch

1. Firstly a Dataloader. Implemented in `dataloader.h`
2. Then the layers and functions in `layers.h`, these include
  - The conv layer, the subsampling layer
  - ReLU, Softmax, and their derivatives for gradient calculation
  - Back Propagation and weight updation for all learnable parameters.
3. `CNN.h`, this consists of the base CNN class, like how you would implement a CNN in Pytorch minus all the useful abstraction.
  - The model architecture must be clearly defined in the correct sequence using the functions or layers defined in `layers.h`
  - All weights must be initialized from a **normal distribution** in random
  - All functions to **reset gradients** must be implemented.
  - All functions required in the forward and backward pass must be ordered and called in Sequence.
  - Weight updation functions must be called as well.
4. `main.cpp` has the training loop.

## dataloader.h

Images are usually in Big-endian.

The first 32-bit integer at the very beginning of a binary file(both images and labels) are called magic numers

Images have a magic number os 2051 `0x000000803`

Text or labels have 2049 `0x000000801`

The MNIST files are stored in big-endian format (most significant byte first), but **most PCs (like x86) use little-endian**. So we need to use `reverseInt` function to convert it into the system's native integer order.

```
int reverseInt(int i) {
    unsigned char c1, c2, c3, c4;
    c1 = i & 255; //Get the last 8 bits
    c2 = (i >> 8) & 255; //Get the 2nd byte
    c3 = (i >> 16) & 255; //Get the 3rd byte
    c4 = (i >> 24) & 255; //Get the 4th byte
    return ((int)c1 << 24) + ((int)c2 << 16) + ((int)c3 << 8) + c4;
    //Reverse the order of bytes
}
```

Now this images is stored as a 1D array in the byte file. An image is made up of pixel and each pixel is a value. Now we need to read the value of each pixel, normalize it so it falls between 0 and 1 and then store it in a image vector.

```

// row*col gives the starting of the row, c is the column offset. this is a
// flattened 1D array representation of 2D image

for(int i = 0; i < number_of_images; i++) {
    data.images[i].resize(data.image_size);
    // MNIST images have one byte per pixel. can be negative as well.
    for(int r = 0; r < n_rows; r++) {
        for(int c = 0; c < n_cols; c++) {
            unsigned char temp = 0;
            file.read((char*)&temp, sizeof(temp));
            data.images[i][r * n_cols + c] = static_cast<float>(temp) /
255.0f; // Normalize pixel values to [0, 1]
        }
    }
    file.close();
    return data;
}

```

Same with labels, labels need to be retrieved from the file and stored in a vector.

## layers.h

Specs

| Layer | Input Size | Weight Size | Bias Size | Output Size |
|-------|------------|-------------|-----------|-------------|
| C1    | 28×28      | 6 × 5×5     | 6         | 6 × 24×24   |
| S1    | 6 × 24×24  | 1 × 4×4     | 1         | 6 × 6×6     |
| F     | 6 × 6×6    | 10 × 6×6×6  | 10        | 10          |

All the layers, activation function, back propagation functions and weight updation functions are available in `layers.h`. It is long so I haven't included code snippets. But in short there are the following functions.

### Activation Functions

- `relu_function` - ReLU activation function (returns  $x$  if  $x > 0$ , else 0)
- `relu_derivative` - ReLU derivative (returns 1 if  $x > 0$ , else 0)
- `softmax_function` - Softmax activation for output layer with numerical stability
- `softmax_cross_entropy_gradient` - Computes gradient for softmax + cross-entropy loss

### Weight Initialization Functions

- `initializeC1Weights` - Initialize convolution layer C1 weights and biases
- `initializeS1Weights` - Initialize subsampling layer S1 weights and biases
- `initializeFWeights` - Initialize fully connected layer weights and biases

## Forward Pass Functions

- `fp_c1` - Forward pass for convolution layer C1 ( $28 \times 28 \rightarrow 6 \times 24 \times 24$ )
- `applyActivationC1` - Apply ReLU activation to C1 layer output
- `fp_s1` - Forward pass for subsampling layer S1 ( $6 \times 24 \times 24 \rightarrow 6 \times 6 \times 6$ )
- `applyActivationS1` - Apply ReLU activation to S1 layer output
- `fp_preact_f` - Forward pass for fully connected layer (flattened input)
- `fp_bias_f` - Add bias to fully connected layer preactivation
- `applyActivationF` - Apply softmax activation to output layer

## Backward Propagation Functions

- `bp_weight_f` - Compute gradients for fully connected layer weights
- `bp_bias_f` - Update fully connected layer biases
- `bp_output_s1` - Backpropagate gradients to S1 output
- `bp_preact_s1` - Compute gradients for S1 preactivation (ReLU derivative)
- `bp_weight_s1` - Compute gradients for S1 layer weights
- `bp_bias_s1` - Update S1 layer bias
- `bp_output_c1` - Backpropagate gradients to C1 output (through pooling)
- `bp_preact_c1` - Compute gradients for C1 preactivation (ReLU derivative)
- `bp_weight_c1` - Compute gradients for C1 layer weights
- `bp_bias_c1` - Update C1 layer biases

## Weight Update Functions

- `updateFWeights` - Update fully connected layer weights using gradients
- `updateS1Weights` - Update S1 layer weights using gradients
- `updateC1Weights` - Update C1 layer weights using gradients

## cnn.h

- Initialize the model weights or Load the model weights from a saved model
- Initialize Gradients
- Define `forward()`, `backward()`, `update_weights()`, `calculateLoss()` functions which are abstractions and to be used in `main.cpp`.
- A function to reset the gradients (important in training loop to avoid gradient accumulation)

## main.cpp

- Run the training loop
- Train the model and save it.
- Implemented Batching, Data Shuffling.
- Has the `trainCNN` function

```
// Clean training function using CNN's built-in methods
void trainCNN(CNN& cnn, const float input[28][28], int target_label) {
    // Reset gradients before each training iteration
    cnn.resetGradients();
    // Forward pass
    cnn.forward(input);
    // Backward pass using CNN's method
    cnn.backward(target_label);
    // Note: We still need to call bp_weight_c1 separately as it needs
    the input
    bp_weight_c1(cnn.d_c1_weight, cnn.d_c1_preact, input);
    // Update weights
    cnn.updateWeights();
}
```

## Observation: Activation Functions & Learning Rate Impact

- Initially implemented the network using sigmoid activation functions throughout, which is historically accurate to early CNN architectures.
  - However, **modern practice suggests using ReLU for hidden layers and softmax for multi-class output.**
  - After switching from sigmoid to ReLU (hidden layers) and softmax (output layer), the model initially performed poorly with very low accuracy (~18%) and high loss values (~2.3).
  - The critical insight was that ReLU networks require a lower learning rate compared to sigmoid networks.
  - Reducing the learning rate from 0.1 to 0.01 dramatically improved performance, achieving 99.70% training accuracy and 96.80% test accuracy within 5 epochs.
  - This demonstrates the **importance of hyperparameter tuning** when changing activation functions, as ReLU's non-saturating gradients can cause instability with learning rates that work fine for sigmoid.
-



# PART-2

## OpenMP Parallelization Report for CudaCNN

### TLDR

This details the comprehensive OpenMP parallelization implemented in the smolCNN project. OpenMP pragmas have been strategically placed to parallelize the most computationally intensive operations while maintaining thread safety and avoiding race conditions.

### OpenMP Implementation Details

#### 1. Forward Pass Parallelization

##### Convolutional Layer 1 (C1)

**Location:** layers.h - fp\_c1() function

```
#pragma omp parallel for
for (int feature = 0; feature < 6; feature++)
```

- **Purpose:** Parallelizes convolution across 6 different feature maps/filters
- **Benefit:** Each thread processes one or more filters independently
- **Computational Impact:** High - convolution is the most expensive operation

##### Activation Functions

**Locations:** layers.h - applyActivationC1() and applyActivationS1()

```
#pragma omp parallel for collapse(2)
for (int feature = 0; feature < 6; feature++)
    for (int row = 0; row < 24; row++)
```

- **Purpose:** Parallelizes ReLU activation across feature maps and rows
- **Benefit:** Loop collapse creates more parallel work items
- **Computational Impact:** Medium - element-wise operations benefit from parallel execution

##### Subsampling/Pooling Layer (S1)

**Location:** layers.h - fp\_s1() function

```
#pragma omp parallel for
for (int feature = 0; feature < 6; feature++)
```

- **Purpose:** Parallelizes pooling operations across feature maps
- **Benefit:** Independent processing of each feature map's pooling operation

## Fully Connected Layer (F)

**Location:** layers.h - fp\_preact\_f() function

```
#pragma omp parallel for
for (int output_neuron = 0; output_neuron < 10; output_neuron++)
```

- **Purpose:** Parallelizes matrix multiplication across output neurons
- **Benefit:** Each thread computes one output neuron independently

## 2. Backward Pass Parallelization

### Fully Connected Layer Backpropagation

**Location:** layers.h - bp\_weight\_f() function

```
#pragma omp parallel for
for (int output_neuron = 0; output_neuron < 10; output_neuron++)
```

- **Purpose:** Parallelizes weight gradient computation
- **Thread Safety:** Each thread updates different weight matrix rows

### Gradient Propagation with Atomic Operations

**Location:** layers.h - bp\_output\_sl() function

```
#pragma omp parallel for collapse(2)
for (int feature = 0; feature < 6; feature++)
    for (int row = 0; row < 6; row++)
        #pragma omp atomic
        d_sl_output[feature][row][col] += ...
```

- **Purpose:** Safely accumulates gradients from multiple output neurons
- **Thread Safety:** Uses atomic operations to prevent race conditions
- **Performance Trade-off:** Atomic operations ensure correctness at slight performance cost

## Activation Derivative Computation

**Locations:** `bp_preact_s1()` and `bp_preact_c1()` functions

```
#pragma omp parallel for collapse(2)
```

- **Purpose:** Parallelizes ReLU derivative computation
- **Benefit:** Independent element-wise operations across feature maps

## Convolutional Weight Gradients

**Location:** `layers.h` - `bp_weight_c1()` function

```
#pragma omp parallel for  
for (int feature = 0; feature < 6; feature++)
```

- **Purpose:** Parallelizes weight gradient accumulation per filter
- **Thread Safety:** Each thread works on different filter weights

## 3. Weight Update Parallelization

### All Layer Weight Updates

**Locations:** `updateFWeights()`, `updateC1Weights()` functions

```
#pragma omp parallel for
```

- **Purpose:** Parallelizes weight update operations
- **Benefit:** Independent updates to different weight matrix sections

## 4. Memory Operations Parallelization

### Gradient Initialization and Reset

**Location:** `cnn.h` - `initializeGradients()` and `resetGradients()`

```
#pragma omp parallel sections  
{  
    #pragma omp section  
    memset(d_c1_weight, 0, sizeof(d_c1_weight));  
    #pragma omp section  
    memset(d_c1_preact, 0, sizeof(d_c1_preact));  
}
```

```
// ... more sections  
}
```

- **Purpose:** Parallelizes memory initialization across different arrays
- **Benefit:** Concurrent memory operations for large gradient arrays

## 5. Data Loading Parallelization

Similar to `num_workers` in Pytorch DataLoader.

### MNIST Data Processing

**Location:** `dataloader.h` - `loadMNISTImages()`

```
#pragma omp parallel for  
for(int pixel_idx = 0; pixel_idx < data.image_size; pixel_idx++)
```

- **Purpose:** Parallelizes pixel normalization within each image
- **Note:** File I/O remains sequential due to file access constraints

## 6. Backprop

The `#pragma omp atomic` forces serialization at each memory location, essentially making the parallel loop run mostly sequentially. This kills performance.

Restructure the loops so each thread works on independent memory locations

**Note:** These are a few important places OpenMP has been used. There are more. Not mentioned to avoid repetition

## Comparison

### Sequential vs OpenMP Performance Comparison Report

#### System Configuration

- **CPU:** 8 cores with hyperthreading (16 logical processors)
- **Architecture:** x86\_64
- **OpenMP Implementation:** GCC OpenMP runtime
- **Dataset:** MNIST (60,000 training images, 10,000 test images)
- **Training Configuration:** 5 epochs, batch size 100, 600 batches per epoch

#### Performance Results

## Execution Time Comparison

| Implementation | Threads | Wall Time (seconds) | CPU Time (user) | Speedup vs Sequential | Efficiency |
|----------------|---------|---------------------|-----------------|-----------------------|------------|
| Sequential     | 1       | 37.095              | 36.823          | 1.00x (baseline)      | 100%       |
| OpenMP         | 2       | 13.816              | 26.110          | 2.68x                 | 134%       |
| OpenMP         | 4       | 11.728              | 45.294          | 3.16x                 | 79%        |
| OpenMP         | 8       | 13.440              | 105.491         | 2.76x                 | 35%        |
| OpenMP         | 16      | 28.354              | 433.240         | 1.31x                 | 8%         |

## Performance Analysis

### Optimal Configuration: 4 Threads

- **Best overall performance** with 3.16x speedup
- Wall time reduced from 37.1s to 11.7s (68% reduction)
- Good balance between parallel efficiency and thread overhead

### Scalability Characteristics

#### Strong Scaling (2-4 threads)

- Nearly linear speedup from 1 to 4 threads
- 2 threads: 2.68x speedup (134% efficiency - superlinear due to cache effects)
- 4 threads: 3.16x speedup (79% efficiency - still very good)

#### Performance Degradation (8+ threads)

- 8 threads: Performance drops to 2.76x speedup (35% efficiency)
- 16 threads: Severe degradation to 1.31x speedup (8% efficiency)

## Key Findings

### 1. Sweet Spot at 4 Threads

- **Maximum speedup** of 3.16x achieved with 4 threads
- **Best wall-clock time** of 11.7 seconds
- **Reasonable efficiency** at 79%

### 2. Diminishing Returns Beyond 4 Threads

- Performance degrades significantly with 8+ threads
- 16 threads perform worse than sequential (1.31x speedup)
- Thread overhead outweighs parallel benefits

### 3. Architecture-Dependent Optimization

- Performance closely tied to physical core count
- Hyperthreading provides minimal benefit for this workload
- Cache hierarchy and memory bandwidth are limiting factors

### 4. Workload Characteristics

- CNN operations are inherently parallel but have limitations
- Atomic operations in backpropagation create bottlenecks
- Matrix operations scale better than element-wise operations

## Conclusion

The OpenMP parallelization successfully achieved significant performance improvements, with the optimal configuration being **4 threads providing a 3.16x speedup**. The implementation demonstrates good understanding of parallel programming principles, though there are opportunities for further optimization, particularly in eliminating atomic operations that cause serialization bottlenecks.

The results clearly show that **more threads do not always equal better performance**, and the optimal configuration depends on the underlying hardware architecture and workload characteristics.

## TO-DO

- Learn Cuda
- Rewrite CNN layers and functions using pointers rather than vectors for Cuda implementation.
- Pytorch cpp with Cuda maybe?